MY AI JOURNEY

From Zero to Building Real AI Systems

Days 1 – 4

A build-first approach to understanding AI — not just in theory, but in practice.

Day 1 — The Beginning

The first step is always the hardest. Not because it requires extraordinary skill, but because we keep waiting for the perfect moment to begin. We tell ourselves: once I have more time, once I feel motivated, once things slow down. And so the days pass.

This time, something was different. I chose not to wait.

Not with textbooks. Not with lengthy video courses. Just with the simple decision to start building — and let understanding follow.

The Approach

Most people start their AI learning journey by diving into heavy theory. Neural networks, backpropagation, gradient descent — all before they have ever run a single line of AI code. That approach tends to stall quickly. The concepts feel abstract, the motivation fades, and the notebook gets closed.

My approach is different. This journey is built on a **build-first mindset**. Learn AI the way it actually works in the real world — by building small, practical projects, experimenting with ideas, and connecting concepts step by step. No unnecessary complexity. Just clear, hands-on learning.

The Roadmap

This is the path that will guide the entire journey:

1. Getting Started with AI

We begin with the basics — understanding AI through simple API calls. Actually interacting with AI through code, not just reading about it.

2. Giving Memory to AI

AI without memory is just a tool. Here, we make it remember context, making it feel smarter and genuinely more useful in conversation.

3. Understanding Embeddings

This is where things start getting interesting. We learn how machines understand meaning — not just words, but the intent behind them.

4. Vector Databases

Once we have embeddings, we need a system to store and search them efficiently. That is exactly where vector databases come in.

5. Working with LangChain

We connect everything together and build structured, intelligent AI workflows.

6. Multi-Agent Systems

Not just one AI — we create multiple agents working together to solve real-world problems.

7. Building Real AI Agents

At the end of this journey, we build powerful AI agents that can actually solve problems, start to finish.

What is AI — In the Simplest Way Possible

Before going deeper, there is one question worth answering properly: what exactly is AI?

At first, it sounds complex. But the idea is surprisingly simple. AI is not magic. It is a system trained on a massive amount of data that can use that data to generate smart-looking answers.

Think of it like this: imagine a person who has read millions of books, watched countless videos, and learned from endless conversations. If you ask that person a question, they respond based on everything they have ever seen and learned. That is exactly how AI works.

What is Happening Behind the Scenes

When you ask AI something, the response feels instant. But in reality, a great deal is happening in the background: massive data centers processing simultaneously, patterns being searched across enormous datasets, and a response being assembled based on probability and relevance. All of this happens within seconds.

What is an LLM

An LLM — a Large Language Model — is the core brain behind modern AI systems. It does not *know* things the way humans do. Instead, it understands patterns in language, predicts the most likely next words, and forms responses that feel meaningful. That is why it seems like it is thinking — but in reality, it is predicting intelligently.

Once you understand this, AI stops feeling overwhelming. You begin to see it as something you can learn, something you can build with, and something you can control.

The Technical Part — Getting Started with AI in Code

At a practical level, we do not build AI models from scratch. Instead, we use APIs provided by existing AI systems. Think of it this way: you are not creating the brain. You are simply communicating with it. Using code, we send a request to an AI model, and it sends back a response. This interaction is the core of modern AI development.

Every AI-based application follows a simple mental model:

Input → Processing → Output

You give it text, and it gives you meaningful text back. Behind the scenes, things are complex — but from a developer's perspective, it is just a request and a response. This changes how we think about development. Instead of writing logic for every small task, you can now generate responses, summarize content, build chat systems, and automate workflows — all by calling an AI API.

Multiple LLM Models — Choosing the Right One

Not all AI models are the same. Each model is trained differently, each has its own strengths, and choosing the right one can make a huge difference in your application. There is no single perfect model — some are better at reasoning, some are optimized for speed, some handle images, and some are built for cost efficiency.

Reasoning and Logical Thinking

Best when a task involves step-by-step problem solving, deep thinking, complex decision making, or understanding tricky questions. They are slower but far more accurate. Examples: GPT-4, Claude 3 Opus, Gemini Advanced. Best for: debugging, logical analysis, multi-step workflows.

General Purpose Models

Balanced models that are good at almost everything — conversations, content generation, basic reasoning, and Q&A. They are fast and cost-effective, making them ideal for most applications. Examples: GPT-4o, Claude 3 Sonnet, Gemini Pro. Best for: chatbots, assistants, daily AI features.

Fast and Lightweight Models

Optimized for speed and low cost. They may not be the most intelligent, but they are extremely efficient. Examples: GPT-4o Mini, Claude 3 Haiku, Gemini Flash. Best for: real-time apps, auto-complete, high-scale systems.

Coding-Focused Models

Trained heavily on code — best at writing, fixing bugs, explaining logic, and generating APIs. Examples: GPT-4, Claude 3 Sonnet, Gemini Pro. Best for: developer tools, code assistants, automation scripts.

Image Generation Models

These models generate visuals from text prompts rather than producing written responses. Examples: DALL·E, Midjourney, Stable Diffusion. Best for: design generation, thumbnails, creative artwork.

Multimodal Models

Advanced models that can handle multiple input types — text, images, and in some cases audio. Examples: GPT-4o, Gemini 1.5 Pro, Claude 3 Sonnet. Best for: image understanding, document parsing, visual AI apps.

Embedding Models

Not for chatting. These convert text into numerical vectors so machines can understand meaning. Examples: OpenAI text-embedding models, Cohere Embeddings. Best for: semantic search, recommendation systems, similarity matching.

Fine-Tuned and Specialized Models

Customized for specific tasks or industries — trained further on focused datasets. Examples: fine-tuned GPT models, domain-specific Gemini models. Best for: healthcare AI, legal AI, industry-specific customer support.

The key question is never *"Which model is the best?"* — it is always ***"What is my use case?"*** Real-world AI systems are built by combining the right tools for the right jobs.

Day 1 Project — Building with the Groq API

Theory only takes you so far. On Day 1, we connected everything with a real project — a simple AI application where the user asks a question and gets a response instantly using the Groq API. This project is the basic foundation of how real AI applications work.

The application flow is straightforward:

1. User enters a message in the frontend.
2. The request is sent to a backend Express API.
3. The backend calls the Groq API.
4. The AI processes the input.
5. The response is streamed back in real time.
6. The user sees the answer appear as it is generated.

Understanding Key Parameters

Tokens are the smallest units of text that the model processes. You are charged based on tokens, and both input and output count. The longer the conversation, the more tokens are used.

Temperature controls how the AI responds. A value of 0 produces strict, predictable answers. A value of 1 produces more creative and varied responses. A setting of 0.7 gives a good balance between logic and creativity — which is what our project uses.

How Streaming Works

Instead of waiting for the entire response to be ready before sending it, streaming lets the AI send small chunks of text as they are generated. The backend forwards each chunk instantly, and the user sees the response appearing in real time — just like watching someone type. This creates a much better user experience than waiting for a full response to load.

Live Demo: https://ag3617502.github.io/instaAI

Day 2 — Giving AI a Memory

Once you can communicate with an AI model, the next question quickly becomes obvious: why does it not remember anything? You tell it your name, and in the very next message, it has already forgotten. You share context about a project, and a few exchanges later, it is as though the conversation never happened.

This is the problem that Day 2 solves.

What is Context?

Context simply means all the previous information that helps AI understand what is happening right now. When you tell an AI your name early in a conversation, and then later ask it to address you personally, it can only do so if that earlier message is still part of the conversation it can see. That history is what we call context.

The reality of most LLMs is that they are **stateless**. Every new request is treated as a completely fresh start. There is no built-in memory. If you want the AI to remember something, you have to include that information explicitly in every request you send.

Two Types of Memory in AI

Short-Term Memory is temporary. It exists only for the duration of a conversation, stored as recent messages, and is limited in size by what is called the context window. Think of it as the last five to ten messages of a chat.

Long-Term Memory is stored permanently — saved in a database and retrievable at any time. This might include user preferences, past conversations, or personalized history.

In the Day 2 project, we focused on short-term memory, because this is what makes a conversation feel natural and coherent.

How We Built a Context-Aware AI System

The solution is elegant in its simplicity. Instead of sending only the current message to the AI, we send the entire recent conversation history along with it. Every message is saved to a MongoDB database. Before calling the AI, we fetch the last few messages from the database, combine them with the current message, and send the full array to the model.

This means the AI now sees not just what you are asking right now, but what you said before and what it replied — giving it all the information it needs to respond intelligently.

The Context Window and Sliding Window

The context window refers to how much past data you send to the AI in each request. You cannot send the entire conversation history indefinitely — token limits and cost constraints make that impractical. Instead, we use a sliding window: keep only the most recent messages, and drop the older ones. This keeps the system fast, cost-efficient, and contextually relevant.

Security and Personalization

We also added user authentication using signup and login, password hashing with Bcrypt, and JWT-based protection. This ensures that each user has their own separate memory, conversations remain private, and the AI experience feels genuinely personalized.

With context in place, the transformation is remarkable. Without it, AI feels disconnected and forgetful. With it, it feels intelligent, aware, and actually useful. That is the difference between a tool and an assistant.

Live Demo: https://ag3617502.github.io/contextAwareAIFE

Day 3 — Understanding Embeddings

Day 3 marks a significant shift. Up until now, we have been working with AI that generates text — responding to prompts, remembering conversations. But to build truly intelligent systems, we need something deeper. We need AI to understand *meaning*.

What are Embeddings?

Embeddings are a way to convert text into numbers — but not random numbers. These numbers represent meaning. Instead of words, AI works with vectors: lists of numbers that capture the semantic content of language.

Consider the words Apple, Mango, and Car. In embedding space, Apple and Mango would be positioned close together, because fruits share similar properties. Car would be far away. The distance between vectors represents the distance between meanings.

From 2D Intuition to Real Vectors

To build intuition, imagine mapping fruits on a two-dimensional chart — one axis for sweetness, another for size. A Grape might land at (2, 2), an Orange at (5, 5), a Watermelon at (9, 9). Fruits that are similar end up near each other. Fruits that are different end up far apart.

Real embeddings work the same way — but instead of 2 dimensions, they use 768, 1024, or even 1536 dimensions. Each dimension represents a hidden feature of the text: its category, usage, tone, or context. We cannot visualize it, but mathematically, the principle is identical.

Comparing Vectors — Cosine Similarity

To determine how similar two pieces of text are, we compare their vectors mathematically. The most common method is cosine similarity, which measures the angle between two vectors. A value close to 1 means the texts are very similar. A value near 0 means they are unrelated. A value near -1 means they are opposite in meaning.

This allows AI to find meaning rather than just matching exact words. A query like *"best backend language"* can correctly match *"Node.js is great for servers"* — even though the exact words are different.

The Embedding Engine Project

The Day 3 project puts all of this into practice. The system takes multiple sentences, converts each into an embedding, then accepts a user query and finds the best match based on semantic similarity. It is essentially a mini search engine powered by meaning rather than keywords.

For example, given three sentences — "I love coding in JavaScript", "The sun is very hot", and "Node.js is great for backend" — and a query of "What should I use for my server?", the system returns the third sentence. Not because the words match exactly, but because the meaning aligns.

This is called semantic search, and it is the foundation of ChatGPT memory, modern search engines, recommendation systems, and AI assistants. Embeddings are like a map — they convert words into positions so AI can measure the distance between meanings.

Live Demo: https://ag3617502.github.io/EmbeddingEngineFE

Day 4 — Vector Databases

If embeddings are the brain's way of understanding meaning, vector databases are the memory system that stores and retrieves that understanding at scale. This is where AI begins handling real-world data — thousands or millions of entries, searched instantly by semantic similarity.

What is a Vector Database?

A vector database is a special type of database that stores data in the form of vectors — embeddings. Instead of storing just the text "Apple is a fruit", it stores the numerical vector that represents the meaning of that text, along with the original content and any relevant metadata.

The Key Difference from Traditional Databases

Traditional databases like MongoDB or MySQL work on exact matches. You query for a record where the name equals a specific string. Vector databases work differently: instead of exact matching, they find the closest meaning. A query for *"backend development"* returns results like *"Node.js is great for servers"* and *"Express is used for APIs"* — even without exact word matches.

Popular Vector Databases

Pinecone

A fully managed cloud solution, known for its speed and reliability in production systems.

ChromaDB

Easy to use and great for beginners, supporting both local and cloud deployments.

Weaviate

An open-source, scalable option that supports hybrid search combining vectors and keywords.

FAISS (by Meta)

High-performance local vector search used for large-scale similarity tasks.

Milvus

An enterprise-level, distributed system designed for extreme scale.

The Day 4 Projects

VectorHub is the basic implementation — a project that takes multiple inputs, converts them into embeddings, stores them in a vector database, and retrieves results based on semantic similarity. This is the foundational step into semantic search.

Live Demo: https://ag3617502.github.io/vectorhub

MultiVectorStore is the advanced version. It adds support for multiple chat sessions, data isolation per user, and metadata filtering — ensuring that one user cannot access another's data. This is the kind of architecture used in real production AI systems.

Live Demo: https://ag3617502.github.io/multivector

Why Vector Databases Matter

Without a vector database, AI applications are limited to simple, stateless chat interactions. With one, you can store thousands of embeddings, search them instantly, build long-term memory systems, and power real use cases like ChatGPT memory, document search, AI assistants, and RAG — Retrieval-Augmented Generation.

Looking Ahead

Four days in, and the foundation is already substantial. We have gone from understanding what AI is, to calling it in code, to giving it memory, to teaching it how to understand meaning, to storing that understanding in a searchable database.

Each concept built on the one before it. That is the power of the build-first approach — every day produces something real, something you can point to and say: I built that.

The journey ahead includes LangChain, multi-agent systems, and eventually — fully autonomous AI agents that can solve complex problems end to end. The foundation is set. The momentum is real.

The best time to start was yesterday. The second best time is now.